



EMERITUS

Learn. From the world's best.

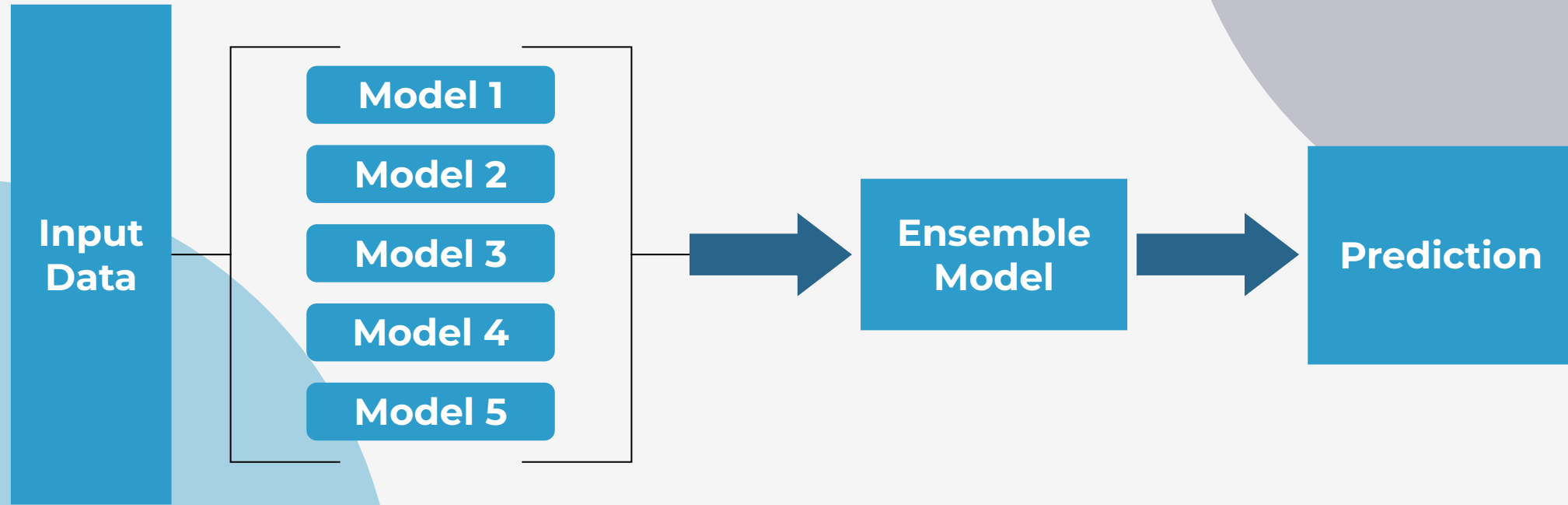
Ensemble Methods



EMERITUS

Learn. From the world's best.

Ensemble Modelling



Voting Classifier

Voting Classifier

Simple and Effective Model Combination

Voting Classifier is a simple ensemble method that merges predictions from multiple models to give a final result

How it works:

1

Train multiple models on the same dataset

2

Generate predictions from each model

3

Combine results using a voting method for the final output

Voting Schemes

Methods to Combine Model Predictions

Hard Voting: Choose the class predicted most often by the models

Soft Voting: Average model probabilities and select the class with the highest score

Advantages & Disadvantages of Voting Classifier

Key Strengths and Limitations

Advantages

- Simple to understand and implement
- Combine models to outperform individual ones
- Reduces overfitting by averaging errors
- Works with various base models

Disadvantages

- Limited gains if models are highly similar
- High computation cost for large datasets

Voting Classifier Implementation

Voting Classifier Implementation

Problem Statement: Wine Quality Classification



Objective: Classify wines into three categories using AdaBoost to boost accuracy by combining weak classifiers



Dataset: UCI Wine dataset with 178 samples and 13 chemical features target three wine varieties — Class 0, 1, and 2

Voting Classifier Implementation

Wine Attributes and Classes

	alcohol	malic_acid	ash	alkalinity_of_ash	magnesium	total_phenols	flavanoids	nonflavanoid_phenols	proanthocyanins	color_intensity	hu
0	14.23	1.71	2.43	15.6	127.0	2.80	3.06	0.28	2.29	5.64	1.0
1	13.20	1.78	2.14	11.2	100.0	2.65	2.76	0.26	1.28	4.38	1.0

Features (13): Alcohol, malic acid, ash, alkalinity of ash, magnesium, total phenols, flavanoids, nonflavanoid phenols, proanthocyanins, colour intensity, hue, OD280/OD315, proline

Target Variable (Classes):

- **class_0:** Wine from cultivar 0
- **class_1:** Wine from cultivar 1

Voting Classifier Implementation

Importing Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.ensemble import VotingClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.metrics import classification_report, confusion_matrix
```

Voting Classifier Implementation

Importing Libraries

Load the Data

- The Wine dataset contains 178 samples with 13 features related to chemical properties.
- The target variable indicates the class of the wine (class_0, class_1, or class_2).

```
wine = load_wine()  
X = wine.data # Features  
y = wine.target # Target variable  
  
# Create a DataFrame for better visualization  
wine_df = pd.DataFrame(data=X, columns=wine.feature_names)  
wine_df['target'] = y
```

Voting Classifier Implementation

Split the Data

- The `train_test_split` function divides the dataset into training (80%) and testing (20%) subsets.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Train the Voting Classifier

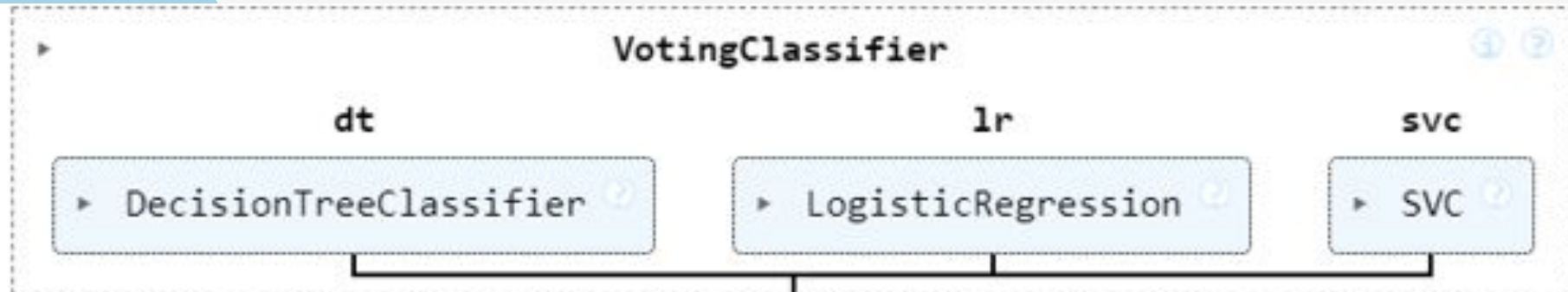
- A Voting Classifier is initialised using three different models: Decision Tree, Logistic Regression and SVM.

Voting Classifier Implementation

Split the Data

```
voting_model = VotingClassifier(  
    estimators=[  
        ('dt', DecisionTreeClassifier(random_state=42)),  
        ('lr', LogisticRegression(max_iter=20000, random_state=42)),  
        ('svc', SVC(probability=True, random_state=42))  
    ],  
    Voting = 'soft' # Use soft voting for better probabilities  
)  
voting_model.fit(X_train, y_train)
```

Out[9]:



Voting Classifier Implementation

Make Predictions and Evaluate the Model

- The predict method generates predictions for both the training and testing datasets.
- The model's performance is evaluated using confusion matrix and classification report.

```
y_train_pred = voting_model.predict(X_train)
y_test_pred = voting_model.predict(X_test)

print("Confusion Matrix: Train Data")
print(confusion_matrix(y_train, y_train_pred))
print("\nClassification Report (Train):")
print(classification_report(y_train, y_train_pred))

print("=====")

print("Confusion Matrix: Test Data")
print(confusion_matrix(y_test, y_test_pred))
print("\nClassification Report (Test):")
print(classification_report(y_test, y_test_pred))
```

Voting Classifier Implementation

Make Predictions and Evaluate the Model

Confusion Matrix: Train Data

```
[[45  0  0]
 [ 0 57  0]
 [ 0  0 40]]
```

Classification Report (Train):

	precision	recall	f1-score	support
0	1.00	1.00	1.00	45
1	1.00	1.00	1.00	57
2	1.00	1.00	1.00	40
accuracy			1.00	142
macro avg	1.00	1.00	1.00	142
weighted avg	1.00	1.00	1.00	142

=====
Confusion Matrix: Test Data

```
[[13  1  0]
 [ 0 14  0]
 [ 0  0  8]]
```

Classification Report (Test):

	precision	recall	f1-score	support
0	1.00	0.93	0.96	14
1	0.93	1.00	0.97	14
2	1.00	1.00	1.00	8
accuracy			0.97	36
macro avg	0.98	0.98	0.98	36
weighted avg	0.97	0.97	0.97	36

Voting Classifier Implementation

Conclusion

Training Data Performance:

- 100% accuracy, all classes (0, 1, 2) with precision, recall, and F1-score of 1.00

Testing Data Performance:

- 97% accuracy overall
- **Class 0:** Precision 1.00, recall 0.93 (missed one case)
- **Class 1:** Precision 0.93, recall 1.00 (some false positives)
- **Class 2:** Perfect precision and recall

Overall Assessment:

- Strong generalisation with minimal errors
- Minor improvement possible for Classes 0 and 1 to address edge cases

Adaboost

Adaboost (Adaptive Boosting)

Boosting Performance with Weak Learners

AdaBoost trains weak learners in sequence, adjusting weights to focus on harder-to-classify examples

Key Features of AdaBoost:

Adaptive weighting: Adjusts weights to focus on difficult cases

Weak learners: Uses weak learners like decision stumps or shallow trees

Ensemble: Combines learners into a stronger ensemble

Simplicity: Simple to understand and implement

Adaboost: How it Works?

Steps to Build the Model

1. **Initialisation:** Assign equal weights to all observations

$$\omega_i = \frac{1}{N}$$

2. **Model Training:** Train a weak learner (e.g., decision stump) on the weighted dataset
3. **Prediction & Error:** Make predictions, calculate weighted error using

$$E = \sum_{i=1}^N \omega_i \cdot \mathbb{I}(y_i \neq h(x_i))$$

Where $\mathbb{I}(y_i \neq h(x_i))$ is an indicator function that equals 1 if the prediction is incorrect and 0 otherwise

Adaboost: How it Works?

Steps to Build the Model

4. **Weight Update:** Increase weights for misclassified cases, decrease for correct ones; calculate learner weight using

$$\alpha_t = 0.5 \times \log \left(\frac{1 - E}{E} \right)$$

Increase weights for misclassified cases; decrease for correct ones.

For misclassified instances

For correctly classified instances:

$$\begin{aligned}\omega_i &= \omega_i \times e^{\alpha t} \\ \omega_i &= \omega_i \times e^{-\alpha t}\end{aligned}$$

5. **Iteration:** Repeat process with updated weights to train new weak learners

6. **Final Model:** Combine all weak learners using their calculated weights

Advantages & Disadvantages of AdaBoost

Strengths and Limitations

Advantages

- **High accuracy:** Often delivers high accuracy
- **Robustness:** Robust to noise and some outliers
- **Interpretability:** Weak learner combination can aid interpretation

Disadvantages

- **Sensitivity to outliers:** Becomes sensitive to outliers if they strongly affect initial weak learners
- **Overfitting:** Cause overfitting if not properly tuned, harming generalisation

Adaboost Implementation

AdaBoost Implementation

Importing Libraries

```
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
import seaborn as sns  
from sklearn.datasets import load_wine  
from sklearn.model_selection import train_test_split  
from sklearn.ensemble import AdaBoostClassifier  
from sklearn.tree import DecisionTreeClassifier  
from sklearn.metrics import classification_report, confusion_matrix
```


AdaBoost Implementation

Importing Libraries

Load the Data

- The Wine dataset contains 178 samples with 13 features, such as alcohol, malic acid, ash, etc.
- The target variable is the class of wine (3 classes: 0, 1, 2).

```
wine = load_wine()
X = wine.data # Features
y = wine.target # Target variable

# Create a DataFrame for better visualization
wine_df = pd.DataFrame(data=X, columns=wine.feature_names)
wine_df['target'] = y
wine_df.head()
```

AdaBoost Implementation

Split the Data

- The `train_test_split` function divides the dataset into training (80%) and testing (20%) subsets.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

AdaBoost Implementation

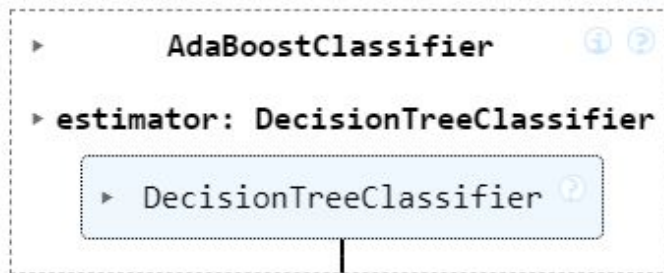
Split the Data

Train the AdaBoost Classifier

- AdaBoost with a Decision Tree base estimator is initialized and trained on the training data.

```
adaboost_model = AdaBoostClassifier(estimator = DecisionTreeClassifier(), n_estimators = 100, random_state = 42)  
adaboost_model.fit(X_train, y_train)
```

Out[11]:



AdaBoost Implementation

Make Predictions and Evaluate the Model

- The predict method generates predictions for both the training and testing datasets.
- The model's performance is evaluated using confusion matrix and classification report.

```
y_train_pred = adaboost_model.predict(X_train)
y_test_pred = adaboost_model.predict(X_test)

print("Confusion Matrix: Train Data")
print(confusion_matrix(y_train, y_train_pred))
print("\nClassification Report (Train):")
print(classification_report(y_train, y_train_pred))

print("=====")

print("Confusion Matrix: Test Data")
print(confusion_matrix(y_test, y_test_pred))
print("\nClassification Report (Test):")
print(classification_report(y_test, y_test_pred))
```

AdaBoost Implementation

Confusion Matrix: Train Data

```
[[45  0  0]
 [ 0 57  0]
 [ 0  0 40]]
```

Classification Report (Train):

	precision	recall	f1-score	support
0	1.00	1.00	1.00	45
1	1.00	1.00	1.00	57
2	1.00	1.00	1.00	40
accuracy			1.00	142
macro avg	1.00	1.00	1.00	142
weighted avg	1.00	1.00	1.00	142

=====
Confusion Matrix: Test Data

```
[[13  1  0]
 [ 0 14  0]
 [ 0  1  7]]
```

Classification Report (Test):

	precision	recall	f1-score	support
0	1.00	0.93	0.96	14
1	0.88	1.00	0.93	14
2	1.00	0.88	0.93	8
accuracy			0.94	36
macro avg	0.96	0.93	0.94	36
weighted avg	0.95	0.94	0.94	36

AdaBoost Implementation

Conclusion Summary

Training Data:

- 100% accuracy
- All samples correctly classified
- Precision, recall, F1-score: 1.00 for all classes

Test Data:

- 94% accuracy
- One Class 1 sample misclassified as Class 0
- One Class 1 sample misclassified elsewhere
- Precision and recall near 1 for most classes

Overall:

- Good generalisation
- Scope to improve class distinction

Gradient Boosting

Gradient Boosting

Sequential Error-Correction in Models

Definition:

Builds models one after another, each correcting previous errors

Purpose:

Enhances accuracy of weak learners, usually decision trees

Gradient Boosting: Key Concepts

Building Models to Reduce Errors

- **Weak Learners:**
 - Use shallow decision trees (stumps)
 - Train each tree to reduce previous residual errors
- **Boosting Process:**
 - **Step 1:** Start with a constant value (e.g., target mean)
 - **Step 2:** Add weak learners to reduce residual error
 - **Step 3** Update model with new predictions, scaled by learning rate

Gradient Boosting: Key Concepts

Building Models to Reduce Errors

- **Learning Rate (η):** Controls each tree's impact; lower values give stronger models but need more trees
- **Loss Function:** Measures model performance; e.g., MSE for regression, Log Loss for classification
- **Regularisation:** Prevents overfitting by limiting tree depth or using subsampling

Boosting Process: Example

Example: Predicting Student Grades (Regression Problem)

Step 1: Dataset

Assume we have a dataset with the following features related to student performance:

Study Hours	Previous Grades	Final Grade
2	C	60
3	B	70
1	D	50
4	A	85
5	B	75

Boosting Process: Example

Step 2: Initialise the Model

Initial Prediction: Start with a simple model, often predicting the mean of the target variable (Final Grade).

Initial prediction (mean grade):

$$\text{Initial Prediction} = (60+70+50+85+75)/5 = 66$$

Boosting Process: Example

Step 3: Compute Residuals

Calculate the residuals, which are the differences between the actual grades and the predicted grades

Final Mark	Initial Prediction	Residual (Actual – Predicted)
60	66	-6
70	66	4
50	66	-16
85	66	19
75	66	9

Boosting Process: Example

Step 4: Fit a Decision Tree to Residuals

Fit a decision tree to the residuals. This tree will learn to predict the residuals instead of the final grades.

- Suppose the decision tree predicts the following adjustments:

Study Hours	Previous Grades	Residual Prediction
2	C	-4
3	B	3
1	D	-10
4	A	15
5	B	5

Boosting Process: Example

Step 5: Update Predictions

Update the initial predictions using the predictions from the decision tree. In Gradient Boosting, the predictions are typically scaled by a learning rate **η (eta)**.

- For example, using $\eta=0.1$

$$\text{Updated Prediction} = \text{Initial Prediction} + \eta \times \text{Residual Prediction}$$

Prediction

Calculating for each student:

Student	Initial Prediction	Residual Prediction	Updated Prediction
1	66	-4	$66 + 0.1 \times (-4) = 65.6$
2	66	3	$66 + 0.1 \times 3 = 66.3$
3	66	-10	$66 + 0.1 \times (-10) = 65.0$
4	66	15	$66 + 0.1 \times 15 = 67.5$
5	66	5	$66 + 0.1 \times 5 = 66.5$

Boosting Process: Example

Step 6: Iterate

- Repeat Steps 3-5 for a specified number of iterations or until the residuals converge.
- Each iteration adds a new decision tree to the ensemble, improving the model's predictions.

Boosting in Classification

Sequentially Improving Classifier Performance

Boosting enhances models by adding classifiers that fix predecessor errors

Similar to Regression Boosting – follows the same process as in regression models

Boosting for Classification

Key Adjustments From Regression Boosting

Target Variable

Predicts categorical class labels, not continuous values

Loss Function

Uses classification-specific loss (e.g., log loss, exponential loss)

Probability Estimates

Outputs class probabilities, converted to labels via a threshold (e.g., 0.5).

Final Prediction

Combines all classifiers' outputs by majority vote or weighted probabilities

Implementation

Often uses decision trees as base learners, focusing on prior misclassifications.

Gradient Boosting – Key Hyperparameters

Main Settings For Tree-based Models

n_estimators – Number of trees to be added

max_depth – Maximum depth of each tree

min_samples_split – Minimum samples to split a internal node

min_samples_leaf – Minimum samples in a leaf node

subsample – Fraction of samples per fit tree (0.5–1.0)

Gradient Boosting – Advantages

Why It's A Powerful Modelling Choice

High accuracy

Delivers strong predictive performance

Versatile

Works with continuous and categorical data

Flexible

Optimises for different loss functions

Gradient Boosting – Limitations

Key Challenges And Considerations

1

Overfitting risk

Especially with many trees;
use regularisation and
early stopping

2

High computational cost

Resource-heavy; opt for
tools like XGBoost or
LightGBM

3

Sensitive to noise

Outliers can hurt
performance; clean and
preprocess data

4

Complex tuning

Requires careful
hyperparameter selection;
use grid/random search

5

Lower interpretability

Harder to explain; use
SHAP for insights



Gradient Boosting Implementation

Gradient Boosting Implementation

Key Attributes For Heart Disease Prediction

- Age
- Sex
- Chest pain type (4 categories)
- Resting blood pressure
- Serum cholesterol (mg/dl)
- Fasting blood sugar (>120 mg/dl)
- Resting ECG results (0, 1, 2)
- Maximum heart rate achieved
- Exercise-induced angina
- ST depression (oldpeak) after exercise
- Slope of peak exercise ST segment
- Major vessels (0–3) seen via fluoroscopy
- Thalassemia (3 = normal, 6 = fixed defect, 7 = reversible defect)
- Target: 1 = heart disease, 0 = no heart disease

Gradient Boosting Implementation

Importing Libraries

- NumPy and Pandas are used for numerical operations and data manipulation.
- Matplotlib and Seaborn are used for visualisation.
- sklearn provides tools for machine learning, including model selection and evaluation metrics.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.preprocessing import StandardScaler
```


Gradient Boosting Implementation

Load the Data

- The Heart Disease dataset contains information about patients, including health measurements and lifestyle factors.
- The target variable indicates whether the patient has heart disease (1) or not (0).
- Load the dataset.

```
heart_df = pd.read_csv("heart_disease.csv")
```

```
#- Data overview  
print(heart_df.head())  
print(heart_df.info())  
print(heart_df.describe())
```

Gradient Boosting Implementation

Load the Data

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	\
0	63	1	3	145	233	1	0	150	0	2.3	0	
1	37	1	2	130	250	0	1	187	0	3.5	0	
2	41	0	1	130	204	0	0	172	0	1.4	2	
3	56	1	1	120	236	0	1	178	0	0.8	2	
4	57	0	0	120	354	0	1	163	1	0.6	2	

Gradient Boosting Implementation

Data Visualisation

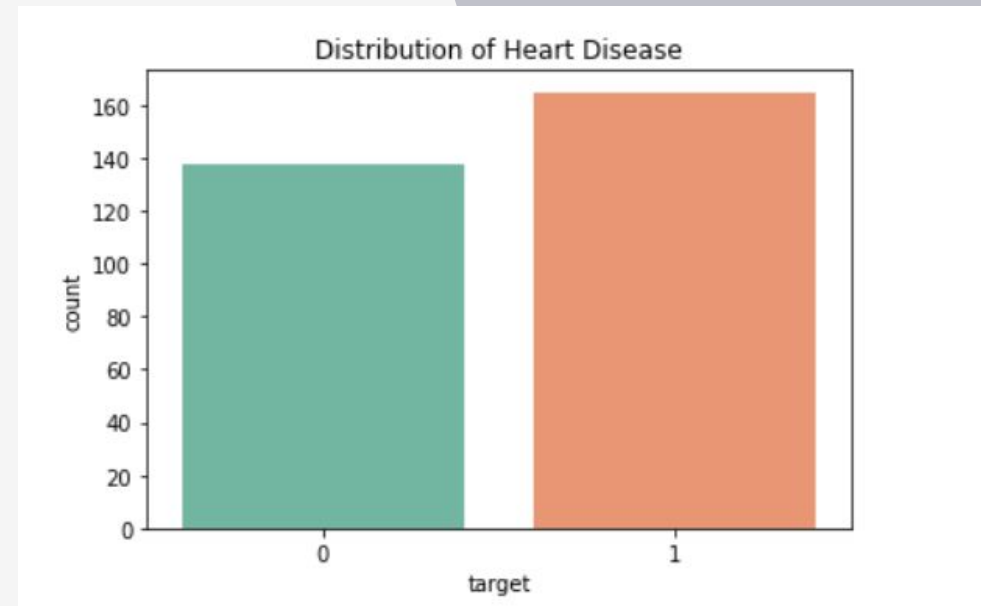
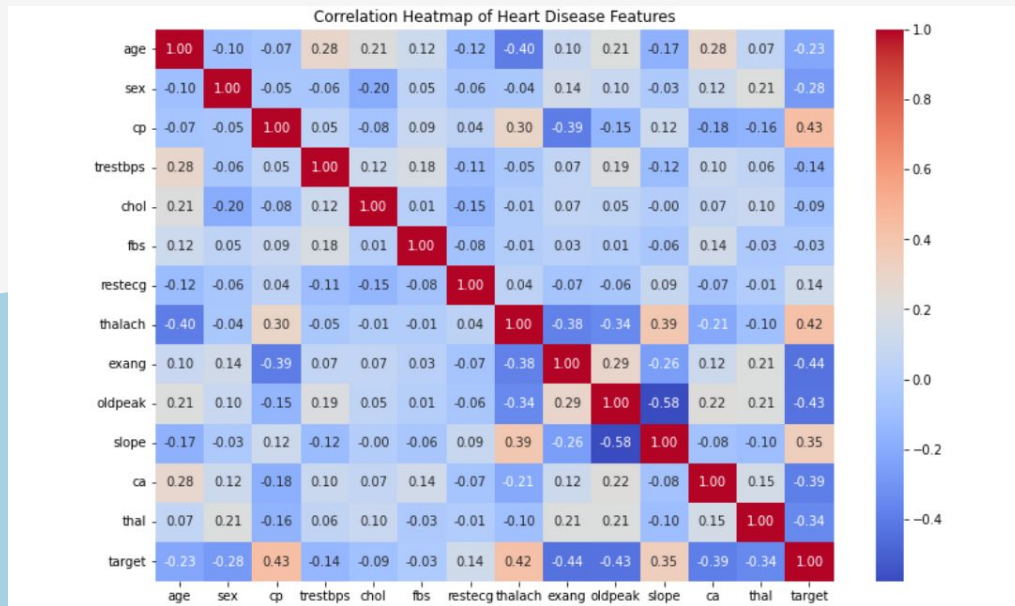
- Before building the model, we will explore the data to understand the distributions of features and relationships between them.
- The heatmap will display the correlation between different features.

```
# Correlation heatmap
plt.figure(figsize=(12, 8))
sns.heatmap(heart_df.corr(), annot=True, cmap='coolwarm', fmt=".2f")
plt.title('Correlation Heatmap of Heart Disease Features')
plt.show()

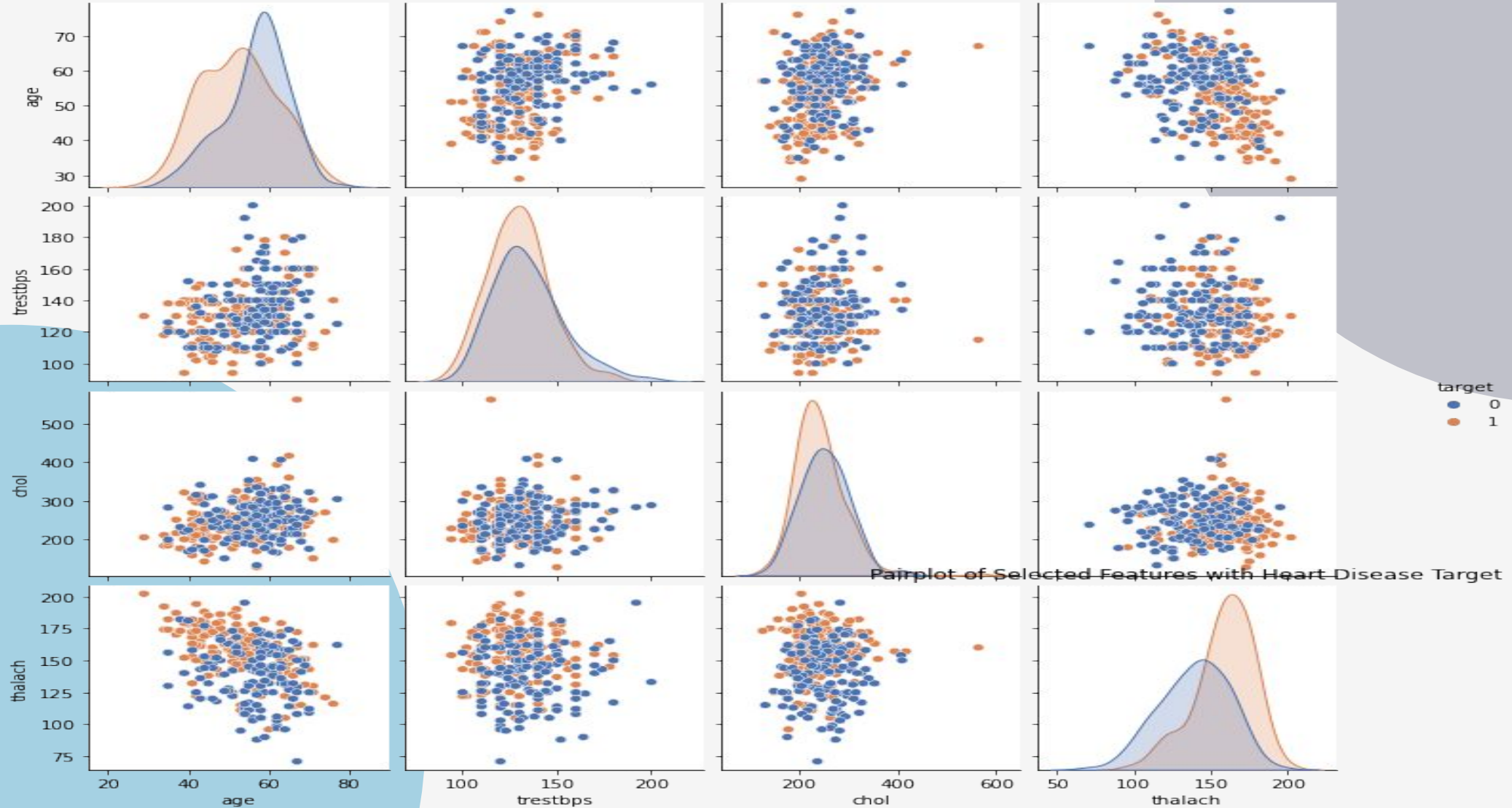
# Distribution of target variable
sns.countplot(x='target', data=heart_df, palette='Set2')
plt.title('Distribution of Heart Disease')
plt.show()

# Pairplot for selected features
sns.pairplot(heart_df[['age', 'trestbps', 'chol', 'thalach', 'target']], hue='target', palette='deep')
plt.title('Pairplot of Selected Features with Heart Disease Target')
plt.show()
```

Gradient Boosting Implementation



Gradient Boosting Implementation



Gradient Boosting Implementation

Split the Data and Scale

- The `train_test_split` function splits the dataset into training (80%) and testing (20%) subsets, ensuring that our model can be evaluated on unseen data.
- Features and target

```
X = heart_df.drop(columns='target')  
y = heart_df['target']
```

```
# Split the data into training and testing sets
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
# Standardize the data (important for Gradient Boosting)
```

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

Gradient Boosting Implementation

Train the Gradient Boosting Model

- Here, we create an instance of the Gradient Boosting classifier and fit it to the training data.
- Gradient Boosting is an ensemble method that combines multiple weak learners (decision trees) to improve accuracy.
- Create a Gradient Boosting classifier.

```
gb_model GradientBoostingClassifier(n_estimators=100, learning_rate=0.1,  
random_state=42)  
gb_model.fit(X_train_scaled, y_train)
```

Out[14]:

```
▼ GradientBoostingClassifier ⓘ ⓘ  
GradientBoostingClassifier(random_state=42)
```

Gradient Boosting Implementation

Make Predictions and Evaluate the Model

- The predict method generates predictions for the test data based on the model trained in the previous step.
- The confusion matrix shows the number of true vs. predicted classifications, allowing us to see where the model makes mistakes.
- The classification report includes precision, recall, and F1-score, providing a comprehensive view of the model's performance.
- Evaluate the model on training data.

```
y_train_pred = gb_model.predict(X_train_scaled)  
y_test_pred = gb_model.predict(X_test_scaled)
```


Gradient Boosting Implementation

Make Predictions and Evaluate the Model

```
## Evaluate the Model
print("Confusion Matrix: Train Data")
print(confusion_matrix(y_train, y_train_pred))
print("\nClassification Report (Train Data):")
print(classification_report(y_train, y_train_pred))
print("=====")

# Evaluate the model on test data
print("Confusion Matrix: Test Data")
print(confusion_matrix(y_test, y_test_pred))
print("\nClassification Report (Test Data):")
print(classification_report(y_test, y_test_pred))
```

Gradient Boosting Implementation

Confusion Matrix: Train Data

```
[[109  0]
 [  0 133]]
```

Classification Report (Train Data):

	precision	recall	f1-score	support
0	1.00	1.00	1.00	109
1	1.00	1.00	1.00	133
accuracy			1.00	242
macro avg	1.00	1.00	1.00	242
weighted avg	1.00	1.00	1.00	242

=====
Confusion Matrix: Test Data

```
[[23  6]
 [ 7 25]]
```

Classification Report (Test Data):

	precision	recall	f1-score	support
0	0.77	0.79	0.78	29
1	0.81	0.78	0.79	32
accuracy			0.79	61
macro avg	0.79	0.79	0.79	61
weighted avg	0.79	0.79	0.79	61

Gradient Boosting Implementation

Feature Importance

- Gradient Boosting provides insights into feature importance, indicating which features contribute most to the model's predictions.
- Plot feature importances

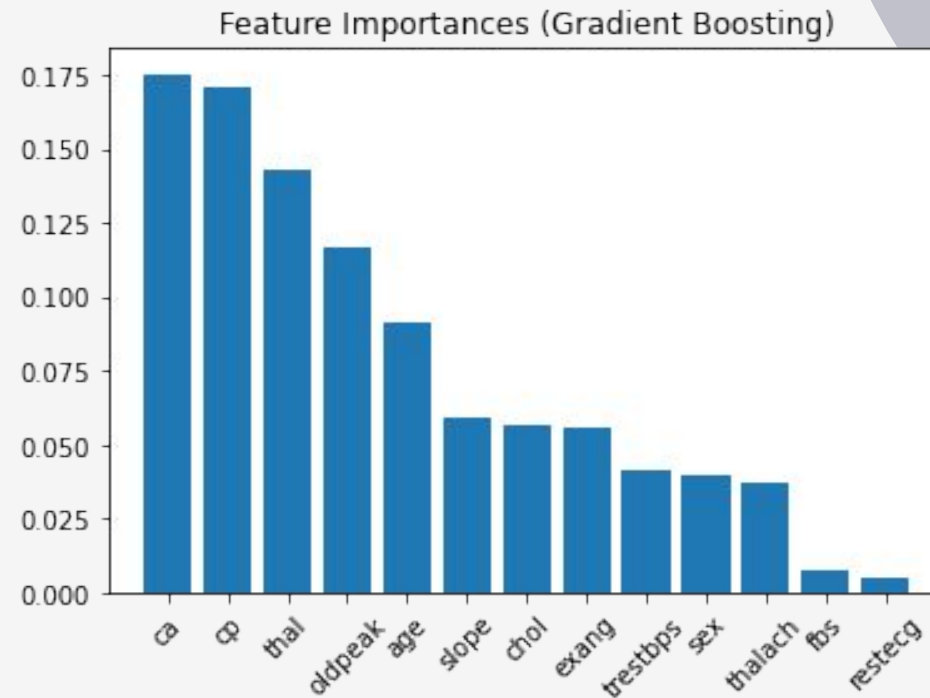
```
importances = gb_model.feature_importances_  
Indices = np.argsort(importances) [::-1]  
  
#- Print feature ranking  
print("Feature ranking:")  
for f in range(X.shape[1]):  
    print(f" {heart_df.columns[Indices [f]]}: {importances [Indices [f]]:.4f}")  
  
#- Plot the feature importances  
plt.figure()  
plt.title("Feature Importances (Gradient Boosting)")  
plt.bar(range (X.shape[1]), importances [Indices], align='center')  
plt.xticks (range (X.shape[1]), np.array (heart_df.columns) [Indices], rotation=45)  
plt.xlim([-1, X.shape[1]])  
plt.show()
```

Gradient Boosting Implementation

Feature Importance

Feature ranking:

ca: 0.1753
cp: 0.1713
thal: 0.1433
oldpeak: 0.1164
age: 0.0914
slope: 0.0594
chol: 0.0568
exang: 0.0557
trestbps: 0.0416
sex: 0.0393
thalach: 0.0372
fbs: 0.0074
restecg: 0.0049



Q&A Session



Open floor for questions and clarifications.